

John the Ripper Laboratory Guide: Password Security and Cracking

Cybersecurity Training Series

November 11, 2024

Legal Disclaimer

This laboratory guide is for educational purposes only. All techniques should be practiced only in authorized, controlled environments.

Contents

1	Laboratory Setup	3
1.1	Environment Requirements	3
1.2	John the Ripper Installation	3
2	Password Creation and Hashing	3
2.1	Creating Password Hashes	3
2.2	Understanding Hash Formats	4
3	Basic Password Cracking	4
3.1	Dictionary Attack	4
3.2	Incremental Mode (Brute Force)	4
4	Advanced Techniques	4
4.1	Custom Rules	4
4.2	Mask Attack	5

5	Practical Exercises	5
5.1	Exercise 1: Basic Password Recovery	5
5.2	Exercise 2: Complex Password Analysis	6
5.3	Exercise 3: Custom Rule Creation	6
6	Password Security Implementation	6
6.1	Creating Secure Passwords	6
6.2	Password Policy Implementation	7
7	Advanced Attack Scenarios	7
7.1	Scenario 1: Corporate Password Audit	7
7.2	Scenario 2: Legacy System Password Recovery	8
8	Performance Optimization	8
8.1	Session Management	8
8.2	Resource Optimization	8
9	Best Practices	9
9.1	Password Security Guidelines	9
9.2	Testing Methodology	9
10	Troubleshooting	9
10.1	Common Issues	9
10.2	Solutions	10

1 Laboratory Setup

1.1 Environment Requirements

- Ubuntu 22.04.5
- 4GB RAM minimum
- 20GB free disk space
- Internet connection
- Root privileges

1.2 John the Ripper Installation

```
1 # Update system
2 sudo apt update && sudo apt upgrade -y
3
4 # Install prerequisites
5 sudo apt install -y build-essential libssl-dev zlib1g-dev
6 sudo apt install -y yasm pkgconf libbz2-dev
7
8 # Clone John the Ripper jumbo version
9 git clone https://github.com/openwall/john.git
10 cd john/src
11
12 # Configure and compile
13 ./configure && make -s clean && make -sj4
```

2 Password Creation and Hashing

2.1 Creating Password Hashes

```
1 # Create a test user password file
2 echo "testuser:password123" > testpass.txt
3
4 # Use OpenSSL to create MD5 hash
5 openssl passwd -1 "password123" > hash.txt
6
7 # Create SHA-512 hash
8 mkpasswd -m sha-512 "password123" > sha512.txt
```

2.2 Understanding Hash Formats

Example hash formats:

- MD5: `1salt`*hash*`SHA-512:6salt`*hash*`bcrypt:2a10salt`*hash*

3 Basic Password Cracking

3.1 Dictionary Attack

```
1 # Download wordlist
2 wget https://github.com/danielmiessler/SecLists/raw/master/
   Passwords/Common-Credentials/10k-most-common.txt
3
4 # Run basic dictionary attack
5 ./john --wordlist=10k-most-common.txt hash.txt
6
7 # Show cracked passwords
8 ./john --show hash.txt
```

3.2 Incremental Mode (Brute Force)

```
1 # Run incremental mode
2 ./john --incremental hash.txt
3
4 # Run specific character set
5 ./john --incremental=Lower hash.txt
6
7 # Show progress
8 ./john --status
```

4 Advanced Techniques

4.1 Custom Rules

Create a custom rule file (rules.conf):

```
1 # Create custom rules
2 echo "[List.Rules:Custom]" > rules.conf
3 echo "Az\"123\" >> rules.conf
```

```

4 echo "Az\"!\\" >> rules.conf
5 echo "A0\"123\" >> rules.conf
6
7 # Apply custom rules
8 ./john --wordlist=wordlist.txt --rules=Custom hash.txt

```

4.2 Mask Attack

```

1 # Create mask for common patterns
2 ./john --mask="?l?l?l?l?d?d" hash.txt
3
4 # Custom character sets
5 ./john --mask="?u?l?l?l?l?d?d?s" hash.txt

```

5 Practical Exercises

5.1 Exercise 1: Basic Password Recovery

Exercise 1

Create and crack a simple password:

```

1 # Create test password
2 echo "user1:$(openssl passwd -1 password123)" > ex1.txt
3
4 # Try to crack it
5 ./john --wordlist=10k-most-common.txt ex1.txt

```

5.2 Exercise 2: Complex Password Analysis

Exercise 2

Create and analyze a complex password:

```
1 # Create complex password
2 echo "user2:$(openssl passwd -1 P@ssw0rd2023!)" > ex2.
  txt
3
4 # Try different attack methods
5 ./john --incremental=Alpha ex2.txt
6 ./john --mask="?u?l?l?l?l?d?d?d?s" ex2.txt
```

5.3 Exercise 3: Custom Rule Creation

Exercise 3

Create custom rules for specific patterns:

```
1 # Create rule file
2 cat << 'EOF' > custom.rules
3 [List.Rules:Custom]
4 # Capitalize first letter
5 c
6 # Add year
7 $2023
8 # Add special char
9 $!
10 EOF
11
12 # Apply custom rules
13 ./john --wordlist=wordlist.txt --rules=Custom ex3.txt
```

6 Password Security Implementation

6.1 Creating Secure Passwords

Python script for secure password generation:

```
1 import random
2 import string
```

```

3
4 def generate_secure_password():
5     length = 16
6     chars = string.ascii_letters + string.digits + string.
      punctuation
7
8     while True:
9         password = ''.join(random.choice(chars) for _ in
      range(length))
10        if (any(c.islower() for c in password)
11            and any(c.isupper() for c in password)
12            and any(c.isdigit() for c in password)
13            and any(c in string.punctuation for c in password
      )):
14            return password
15
16 # Generate 5 secure passwords
17 for _ in range(5):
18     print(generate_secure_password())

```

6.2 Password Policy Implementation

Python script for password policy checking:

```

1 def check_password_strength(password):
2     score = 0
3     checks = {
4         'length': len(password) >= 12,
5         'uppercase': any(c.isupper() for c in password),
6         'lowercase': any(c.islower() for c in password),
7         'digits': any(c.isdigit() for c in password),
8         'special': any(not c.isalnum() for c in password)
9     }
10
11     return sum(checks.values()), checks

```

7 Advanced Attack Scenarios

7.1 Scenario 1: Corporate Password Audit

```

1 # Create password policy test file
2 cat << 'EOF' > corp_policy.rule

```

```

3 # Must contain uppercase
4 u
5 # Must contain number
6 $[0-9]
7 # Must contain special char
8 $[!@#%$]
9 EOF
10
11 # Audit passwords
12 ./john --wordlist=corporate_dict.txt --rules=corp_policy
    hash_dump.txt

```

7.2 Scenario 2: Legacy System Password Recovery

```

1 # Create old-style password hashes
2 # DES format
3 openssl passwd -crypt "oldpassword" > old_hash.txt
4
5 # Crack with specific format
6 ./john --format=crypt old_hash.txt

```

8 Performance Optimization

8.1 Session Management

```

1 # Start a named session
2 ./john --session=audit1 hash.txt
3
4 # Resume session
5 ./john --restore=audit1

```

8.2 Resource Optimization

```

1 # Set number of threads
2 ./john --fork=4 hash.txt
3
4 # Use OpenMP
5 ./john --unit=12 hash.txt

```


9 Best Practices

9.1 Password Security Guidelines

- Use minimum 12 characters
- Mix character types
- Avoid common patterns
- Use unique passwords
- Implement MFA where possible

9.2 Testing Methodology

1. Start with simple dictionary attacks
2. Progress to rule-based attacks
3. Use incremental mode as last resort
4. Document all findings
5. Implement improvements

10 Troubleshooting

10.1 Common Issues

- Hash format detection issues
- Performance problems
- Memory limitations
- Session corruption

10.2 Solutions

```
1 # Fix hash format issues
2 ./john --list=formats
3 ./john --format=raw-md5 hash.txt
4
5 # Fix performance issues
6 ./john --unit=8 hash.txt
```